

End-Effector Tool gripper configuration

gripperControl

The end effector of RealMan robotic arms supports various common grippers. In the system, the INSPIRE-ROBOTS EG2-4C2 gripper is adapted for use. To facilitate user operation of the gripper, the robotic arm controller provides an open gripper control protocol (the gripper control protocol is mutually exclusive with the end effector modbus functionality). This interface is used to control the stroke, opening and closing, grasping position, and grasping force, and to read the state of the EG2-4C2 gripper.

Set gripper stroke `rm_set_gripper_route()`

Set gripper stroke, which refers to the maximum and minimum opening values of the gripper. Once set successfully, the values are saved automatically and will not be lost when the gripper is powered off.

- **Method prototype:**

```
int rm_set_gripper_route(rm_robot_handle * handle,int min_limit,int max_limit)
```

C

Jump to [rm_robot_handle](#) for details of the structure.

- **Parameter description:**

Parameter	Type	Description
<code>handle</code>	Input	Robotic arm handle.
<code>min_limit</code>	Input	Minimum opening value of the gripper, range: 0-1,000, without unit of measurement.
<code>max_limit</code>	Input	Maximum opening value of the gripper, range: 0-1,000, without unit of measurement.

- **Return value:**

0 represents success. For other error codes, please refer to the [API2 Error Codes](#).

- **Usage demo**

```
//Set the minimum opening value of the gripper to 70 and the maximum value to 500
int min_limit = 70;
int max_limit = 500;
ret=rm_set_gripper_route(robot_handle,min_limit,max_limit);
```

Release gripper `rm_set_gripper_release()`

Release the gripper, which means that the gripper moves to the maximum opening position at the specified speed.

- **Method prototype:**

```
int rm_set_gripper_release(rm_robot_handle * handle,int speed,bool block,int timeout)
```

Jump to [rm_robot_handle](#) for details of the structure.

- **Parameter description:**

Parameter	Type	Description
<code>handle</code>	Input	Robotic arm handle.
<code>speed</code>	Input	Release speed of the gripper, range: 1-1,000, without unit of measurement.
<code>block</code>	Input	true indicates blocking mode, where the system waits for the controller to return the gripper's in-position command; false indicates non-blocking mode, where no in-position command is received.
<code>timeout</code>	Input	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0 - immediately return after sending the command; other values – return after receiving the successful setting command.

- **Return value:**

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version

Parameter	Type	Description	Handling Suggestions
			compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo

C

```
//The gripper releases at a speed of 500. If no response is received within 10 second
int speed =500;
ret = rm_set_gripper_release(robot_handle, speed, true, 10);
```

Gripper force-controlled grasping `rm_set_gripper_pick()`

Gripper force-controlled grasping: The gripper grasps with the set speed and force. When the gripping force exceeds the preset threshold, the grasping operation stops.

- **Method prototype:**

C

```
int rm_set_gripper_pick(rm_robot_handle * handle,int speed,int force,bool block,int
```

Jump to [rm_robot_handle](#) for details of the structure.

- **Parameter description:**

Parameter	Type	Description
<code>handle</code>	Input	Robotic arm handle.
<code>speed</code>	Input	Release speed of the gripper, range: 1-1,000, without unit of measurement.
<code>force</code>	Input	Force control threshold, range: 50-1,000, without unit of measurement.
<code>block</code>	Input	true indicates blocking mode, where the system waits for the controller to return the gripper's in-position command; false indicates non-blocking mode, where no in-position command is received.
<code>timeout</code>	Input	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0 - immediately return after sending the command; other values – return after receiving the successful setting command.

- **Return value:**

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version

Parameter	Type	Description	Handling Suggestions
			compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo

C

```
//Set the grasping speed to 500, and the torque threshold to 200. If no response is
int speed = 500;
int force = 200;
ret = rm_set_gripper_pick(robot_handle,speed,force,true,10);
```

Gripper continuous force-controlled grasping `rm_set_gripper_pick_on()`

- Method prototype:

C

```
int rm_set_gripper_pick_on(rm_robot_handle * handle,int speed,int force,bool block,i
```

Jump to [rm_robot_handle](#) for details of the structure.

- Parameter description:

Parameter	Type	Description
<code>handle</code>	Input	Robotic arm handle.
<code>speed</code>	Input	Release speed of the gripper, range: 1-1,000, without unit of measurement.
<code>force</code>	Input	Force control threshold, range: 50-1,000, without unit of measurement.
<code>block</code>	Input	true indicates blocking mode, where the system waits for the controller to return the gripper's in-position command; false indicates non-blocking mode, where no in-position command is received.
<code>timeout</code>	Input	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0 - immediately return after sending the command; other values – return after receiving the successful setting command.

- Return value:

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version

Parameter	Type	Description	Handling Suggestions
			compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description . ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo

C

```
//Set the grasping speed to 500, and the grasping torque threshold to 200. If no res
int speed = 500;
int force = 200;
ret = rm_set_gripper_pick_on(robot_handle,speed,force,true,10);
```



Set gripper to a specified position

rm_set_gripper_position()

Gripper reaches specified position: When the current opening is smaller than the specified opening, the gripper releases at the specified speed to reach the desired position. When the current opening is larger than the specified opening, the gripper closes at the specified speed and torque to reach the target position. The gripper stops when the gripping force exceeds the torque threshold or when the specified position is reached.

- **Method prototype:**

C

```
int rm_set_gripper_position(rm_robot_handle * handle,int position,bool block,int tim
```

Jump to [rm_robot_handle](#) for details of the structure.

- **Parameter description:**

Parameter	Type	Description
handle	Input	Robotic arm handle.
position	Input	Opening position of the gripper, range: 1-1,000, without unit of measurement.
block	Input	true indicates blocking mode, where the system waits for the controller to return the gripper's in-position command; false indicates non-blocking mode, where no in-position command is received.
timeout	Input	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0 - immediately return after sending the command; other values – return after receiving the successful setting command.

• Return value:

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware

Parameter	Type	Description	Handling Suggestions
			version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo

C

```
//Control the gripper to reach an opening of 500. If no response is received within
int position = 500;
ret = rm_set_gripper_position(robot_handle,position,true,10);
```

##Query gripper state `rm_get_gripper_state()`

• Method prototype:

C

```
int rm_get_gripper_state(rm_robot_handle * handle,rm_gripper_state_t * state)
```

Jump to [rm_robot_handle](#) and [rm_gripper_state_t](#) for details of the structure.

• Parameter description:

Parameter	Type	Description
<code>handle</code>	Input	Robotic arm handle.
<code>state</code>	Output	Store the gripper status structure.

• Return value:

0 represents success. For other error codes, please refer to the [API2 Error Codes](#).

WARNING

By default, this interface does not update data. State updates are enabled starting from the first control of the gripper. However, if the dexterous hand is controlled or the end effector modbus functionality is activated, data updates will be disabled. Additionally, the gripper must support the latest firmware to enable this feature.

• Usage demo

C

```
//Get gripper state
rm_gripper_state_t state;
ret = rm_get_gripper_state(handle, &state);
```

Last Update Time: 2025/4/10 14:27:14

Previous

[Global Waypoint Management](#)

Next

[Five-Finger Dexterous Hand Configuration](#)